

Hiring Assessment - AI Generated song detection

AI-Original Pairwise Similarity

Objective: This assessment evaluates the candidate's ability to design and implement a robust **pairwise similarity metric** to detect and quantify the relationship between an original human-made track and a potential AI-generated counterpart. Instead of simple binary classification, the focus is on **relational modeling**: determining if Track B is a synthetic derivative, variation, or "hallucinated" generation based on Track A.

Goal: Develop a system that takes two full-length audio files as input and outputs a **Similarity (Attribution) Score**. This score must reflect the likelihood that the synthetic track is an attribution of the original, moving beyond short-clip matching to account for variations in tempo, arrangement, and spectral artifacts characteristic of modern generators like Suno and Udio.

Dataset Overview:

Candidates should leverage a multi-source approach to ensure high-variance training and testing:

- **SONICS Dataset:** The primary source for large-scale, end-to-end synthetic songs (97k+ tracks) generated by platforms like Suno and Udio ([👤 awsaf49/sonics · Datasets at Hugging Face](#)).
- **FakeMusicCaps Dataset:** A specialized dataset for analyzing text-to-music (TTM) outputs, containing 10-second clips regenerated via various state-of-the-art models for fine-grained artifact analysis ([🔗 FakeMusicCaps](#)).
- **MIPPIA Dataset (SMP):** Essential for the attribution task and evaluating melodic similarity.

Note: The MIPPIA dataset (Similar Music Pair) provides a dedicated **download function** (`download.py`) to fetch the original high-quality audio files from their source URLs, that can be used for the other datasets as well ([🔗 GitHub - Mippia/smp_dataset](#)), libraries needed: 'pandas', 'yt-dlp' and installation of 'ffmpeg' .

Assessment Instructions:

- **Data Exploration:** Analyze the dataset and provide insights.
- **Feature Engineering:** Propose and justify the use of specific audio or not features. Consider whether standard features (MFCCs, Mel-spectrograms) are sufficient or if higher-order features (Phase-Continuity, Harmonic-to-Noise Ratio) are required.

- **Model Architecture:** Design a solution capable of handling variable-length audio (avg. 3 minutes). Candidates should address how they prevent "window-bias" (only looking at the first 30 seconds).
 - **Bonus: Attribution Logic:** Implement a similarity metric to evaluate the relationship between pairs of tracks.
-

Deliverables

- A Python-based prototype implementing the agent and tools (script or notebook)
 - CLI or function to run: `compare_tracks(song_id_1, song_id_2)`
 - Sample outputs and evaluations (e.g., comparisons between known original/AI Cover and unrelated songs)
 - A short report describing:
 - Solution architecture
 - Tools and models used
 - Design decisions and trade-offs
 - Limitations and ideas for future improvements
-

Submission

- Upload your implementation to a public or private GitHub repository
 - Include:
 - `README.md` with setup and usage instructions
 - Annotated code for tools and agent logic
 - PDF report
-

Encouraged Libraries & Tools

- **Audio:** Librosa, Essentia, TorchAudio, CLAP, MERT
- **Similarity:** Scikit-learn, FAISS, cosine similarity, JaroWinkler
- **Models:** PyTorch, TensorFlow, Scikit-Learn, Hugging Face